F-Code
Code the dream

QUẢN LÝ THÀNH VIÊN VI PHẠM CLB F-CODE

F-Code Violation Management System

Tên đề tài	QUẢN LÝ THÀNH VIÊN VI PHẠM CLB F-CODE
Ngôn ngữ lập trình	C
Đơn vị tổ chức	Ban Học Thuật CLB F-Code, Trường ĐH FPT TP.HCM

Thời gian thực hiện	17/04/2026 đến 17/05/2026
Mentor phụ trách	
Tên nhóm	
Thành viên nhóm	

Đề tài được phát triển bởi ***Phạm Hoàng Tuấn***

Đại diện Ban Học Thuật CLB F-Code, Trường ĐH FPT TP. HCM

Mục lục

Mục lục 2
Phần 1. Mô tả đề tài 3
1.1. Bài toán 3
1.2. Mục tiêu 3
1.3. Phạm vi 3
Phần 2. Yêu cầu chức năng 5
Phần 3. Gợi ý tổ chức thư mục & file mã nguồn 8
Lưu ý khi tổ chức code 8
Phần 4. Gợi ý thiết kế cấu trúc dữ liệu 9
4.1. Struct Member - thông tin thành viên 9
4.2. Struct Violation - thông tin vi phạm 9
Phần 5. Tiêu chí đánh giá 11
5.1. Chức năng đầy đủ và chạy đúng - 30 điểm 11
5.2. Tổ chức code và cách đặt tên - 20 điểm 11
5.3. Lịch sử commit trên Git/GitHub - 15 điểm 11
5.4. Demo chương trình chạy thực tế - 20 điểm 11
5.5. Vấn đáp - hiểu code của chính mình - 15 điểm 11

Phần 1. Mô tả đề tài

1.1. Bài toán

CLB F-Code hiện có hàng trăm thành viên hoạt động xuyên suốt các kỳ học, phân bố trong 4 ban: **Planning**, **HR**, **Media**, **Academic**. Mỗi thành viên mặc định thuộc ban **Academic** khi tham gia vào CLB F-Code và cũng có thể trở thành thành viên của những ban khác.

CLB có bộ nội quy rõ ràng: mặc áo vào thứ Ba và sự kiện, đi họp đúng giờ, tham gia tối thiểu 1 hoạt động mỗi kỳ, không sử dụng bạo lực. Nhưng trên thực tế, không ai biết ai đã vi phạm bao nhiêu lần - vì toàn bộ việc ghi nhận đang nằm trong tin nhắn Zalo và trí nhớ của HR.

Hệ quả rất cụ thể: cuối kỳ không biết ai nợ tiền phạt, ai sắp đủ điều kiện Out CLB vì vắng họp liên tiếp. Ban Chủ Nhiệm muốn xử lý cũng không có dữ liệu để ra quyết định minh bạch.

Nhiệm vụ của nhóm là xây dựng chương trình giải quyết đúng bài toán đó, chạy hoàn toàn trên terminal, không cần giao diện đồ họa, kết nối mạng, cơ sở dữ liệu, không dùng thư viện ngoài thư viện C chuẩn.

Ràng buộc của bài toán:

- Hệ thống hỗ trợ tối đa 1.000 thành viên tại một thời điểm
- Mỗi thành viên định danh duy nhất bằng MSSV, không được trùng
- Mức phạt tiền: Member là 20.000đ/lần, Leader / Vice / Ban Chủ Nhiệm là 50.000đ/lần
- Vắng họp quá 3 buổi liên tiếp không phép sẽ Out CLB
- Vi phạm bạo lực sẽ Out CLB ngay, không áp dụng phạt tiền
- Dữ liệu phải được lưu vào file sau mỗi thao tác, đảm bảo không mất khi tắt chương trình

1.2. Mục tiêu

Sau khi hoàn thành, nhóm sẽ thực hành được:

- Dùng **struct** để lưu thông tin thành viên và vi phạm
- Dùng **mảng** để quản lý danh sách thành viên
- Đọc và ghi file để dữ liệu không bị mất khi tắt chương trình
- Chia chương trình thành nhiều file .h và .c cho gọn, dễ bảo trì
- Làm việc nhóm với Git, commit đều đặn và có ý nghĩa

1.3. Phạm vi

Bao gồm:

- Quản lý thông tin thành viên CLB (thêm, sửa, xóa, tìm kiếm)
- Ghi nhận vi phạm theo từng lần với lý do cụ thể
- Tự động tính mức xử lý theo chức vụ và loại vi phạm

- Lưu dữ liệu vào file, đọc lại khi khởi động
- Thống kê tổng tiền phạt đã thu và chưa thu trong kỳ

Phần 2. Yêu cầu chức năng

STT	Chức năng	Mức độ	Ghi chú
Nhóm 0 – Xác thực & Phân quyền (Authentication)			
0.1	Đăng nhập hệ thống	**Bắt buộc**	Nhập MSSV + mật khẩu. Xác định role, load menu tương ứng. Sai 3 lần → khóa phiên, thoát chương trình
0.2	Đăng xuất	**Bắt buộc**	Xóa session hiện tại, quay về màn hình đăng nhập. Không thoát chương trình
0.3	Đổi mật khẩu	**Bắt buộc**	Thành viên đổi mật khẩu của chính mình (cần nhập mật khẩu cũ). BCN có thể reset mật khẩu bất kỳ thành viên
Nhóm 1 - Chức năng của Thành viên thường (Member Role)			
1.1	Xem profile cá nhân	**Bắt buộc**	Hiển thị: họ tên, MSSV, email, SĐT, ban, chức vụ, trạng thái tân thành viên. Không được sửa trực tiếp
1.2	Xem lịch sử vi phạm của bản thân	**Bắt buộc**	Chỉ xem vi phạm của chính mình: lý do, ngày, mức phạt, trạng thái đã thu / chưa thu. Không xem được vi phạm người khác
1.3	Xem tổng tiền phạt còn nợ	**Bắt buộc**	Tính tổng các vi phạm có isPaid = 0.

			Hiển thị rõ từng khoản và tổng cộng
1.4	Xem danh sách thành viên CLB	**Bắt buộc**	Chỉ hiển thị: họ tên, MSSV, ban, chức vụ. Không hiển thị SĐT, email, số lần vi phạm của người khác
Nhóm 2 - Chức năng của Ban Chủ Nhiệm (Admin Role)			
2.1	Thêm thành viên mới	**Bắt buộc**	Nhập: họ tên, MSSV, email, SĐT, ban, chức vụ, mật khẩu mặc định. Validate MSSV không trùng, email hợp lệ. Tự động tạo tài khoản trong accounts.dat
2.2	Sửa thông tin thành viên	**Bắt buộc**	Sửa được tất cả trừ MSSV. Nếu thay đổi chức vụ → cập nhật lại mức phạt cho các vi phạm chưa thu tiền
2.3	Xóa thành viên	**Bắt buộc**	Xóa thành viên + lịch sử vi phạm + tài khoản đăng nhập. Yêu cầu xác nhận lại trước khi thực hiện xóa
2.4	Ghi nhận vi phạm	**Bắt buộc**	Chọn thành viên (tìm theo MSSV/tên), chọn lý do, hệ thống tự ghi thời gian và tính mức phạt. Lý do bạo lực → cảnh báo và confirm Out CLB
2.5	Đánh dấu đã thu tiền phạt	**Bắt buộc**	Chọn từng vi phạm để đánh dấu

			isPaid = 1. Cập nhật totalFine của thành viên ngay lập tức
2.6	Xem danh sách vi phạm toàn CLB	**Bắt buộc**	Hiển thị toàn bộ vi phạm, lọc theo: ban, lý do vi phạm, trạng thái thu tiền
2.7	Thống kê tiền phạt theo ban	**Bắt buộc**	Tổng đã thu / chưa thu của từng ban. Hiển thị dạng bảng trên terminal
2.8	Cảnh báo thành viên ngưỡng Out CLB	**Bắt buộc**	Tự động kiểm tra khi ghi vi phạm: consecutiveAbsences >= 2 → cảnh báo. Nếu = 3 → thông báo Out CLB, yêu cầu BCN xác nhận xử lý
2.9	Sắp xếp danh sách theo số lần vi phạm	**Nâng cao**	Tăng / giảm. Thuật toán tự cài, không dùng qsort
2.10	Xuất báo cáo vi phạm ra file .txt	**Nâng cao**	Có timestamp, tổng hợp theo ban, danh sách thành viên còn nợ tiền phạt
2.11	Tìm kiếm vi phạm theo khoảng thời gian	**Tùy chọn**	Nhập ngày bắt đầu - kết thúc, lọc theo violationTime bằng time_t

Bắt buộc - Phải có đầy đủ trước khi nộp Final Report.

Nâng cao - Có điểm cộng và có thể phát triển tính năng khác ngoài gợi ý nếu cảm thấy hợp lý.

Tùy chọn - Không bắt buộc.

Phần 3. Gợi ý tổ chức thư mục & file mã nguồn

File / Thư mục	Vai trò
main.c	Khởi động chương trình, hiển thị menu chính theo role sau khi đăng nhập
auth.h / .c	Đăng nhập, đăng xuất, đổi mật khẩu, kiểm tra quyền truy cập
member.h / .c	Thêm, sửa, xóa, tìm kiếm thành viên
violation.h / .c	Ghi nhận vi phạm, tinh muc xu ly, canh bao Out CLB
fileio.h / .c	Đọc và ghi dữ liệu ra file (members.dat, violations.dat, accounts.dat)
report.h / .c	Thống kê tiền phạt, xuất báo cáo vi phạm
utils.h / .c	Các hàm dùng chung: kiểm tra MSSV, validate email, định dạng thời gian
data/members.dat	File lưu dữ liệu thành viên (đọc lại mỗi lần khởi động)
data/violations.dat	File lưu lịch sử vi phạm (đọc lại mỗi lần khởi động)
data/accounts.dat	File lưu tài khoản đăng nhập: MSSV, mật khẩu, role, trạng thái khóa

Lưu ý khi tổ chức code

- **main.c** chỉ nên chứa menu và điều hướng - không viết toàn bộ logic vào một file duy nhất
- Mỗi file .c nên có file .h đi kèm để khai báo hàm dùng ở nơi khác
- Ưu tiên làm member.h và violation.h trước. Nếu chưa quen, bắt đầu với 2-3 file rồi tách dần theo tiến độ
- Đặt tên biến và hàm bằng tiếng Anh, chuẩn quy ước chung và nhất quán trong toàn bộ project

Phần 4. Gợi ý thiết kế cấu trúc dữ liệu

4.1. Struct Member - thông tin thành viên

Mỗi thành viên CLB có thể được lưu bằng một struct như sau:

Tên trường	Kiểu dữ liệu	Lưu thông tin gì
fullName	char[]	Họ và tên - VD: Pham Hoang Tuan
email	char[]	Email - VD: tuanph@fpt.edu.vn
phone	char[]	Số điện thoại
studentId	char[]	MSSV - VD: SE200000 (định danh duy nhất, không cho phép sửa)
team	int	Ban: 0 = Academic, 1 = Planning, 2 = HR, 3 = Media
role	int	Chức vụ: 0 = Member, 1 = Leader/Vice, 2 = Ban Chủ Nhiệm
violationCount	int	Tổng số lần vi phạm (tự động tăng khi ghi vi phạm mới)
consecutiveAbsences	int	Số buổi họp vắng liên tiếp (dùng để cảnh báo Out CLB)
totalFine	double	Tổng tiền phạt tích lũy (tính lại sau mỗi lần cập nhật isPaid)

4.2. Struct Violation - thông tin vi phạm

Mỗi lần vi phạm được ghi lại bằng một struct như sau:

Tên trường	Kiểu dữ liệu	Lưu thông tin gì
studentId	char[]	MSSV người vi phạm - khớp với Member.studentId
reason	int	0 = Không mặc áo CLB, 1 = Vắng họp, 2 = Không tham gia hoạt động, 3 = Bạo lực
violationTime	time_t	Thời điểm ghi nhận vi phạm (tự động lấy từ time())

fine	double	Member = 20.000đ, Leader/Vice/BCN = 50.000đ, bạo lực = 0đ
isPaid	int	Trạng thái thu tiền: 0 = Chưa thu, 1 = Đã thu
penalty	int	Hình thức xử lý cuối: 0 = Phạt tiền, 1 = Out CLB
note	char[]	Ghi chú thêm (tùy chọn, có thể để trống)

4.3. Struct Account - thông tin tài khoản đăng nhập

Mỗi tài khoản đăng nhập được lưu trong accounts.dat với cấu trúc sau:

Tên trường	Kiểu dữ liệu	Lưu thông tin gì
studentId	char[]	MSSV - khớp với Member.studentId, dùng để tra cứu thông tin sau khi đăng nhập
password	char[]	Mật khẩu (lưu plain text - đủ với C cơ bản, không cần mã hóa)
role	int	0 = Member, 1 = Ban Chủ Nhiệm (BCN)
isLocked	int	1 = Tài khoản bị khóa sau 3 lần đăng nhập sai liên tiếp
failCount	int	Số lần đăng nhập sai liên tiếp (reset về 0 khi đăng nhập thành công)

Lưu ý: Khi BCN tạo thành viên mới (chức năng 2.1), hệ thống tự động tạo bản ghi Account tương ứng với mật khẩu mặc định là MSSV của thành viên đó.

Phần 5. Tiêu chí đánh giá

Tiêu chí đánh giá	Điểm tối đa
Chức năng đầy đủ và chạy đúng	30
Tổ chức code và cách đặt tên	20
Lịch sử commit trên Git/GitHub	15
Demo chương trình chạy thực tế	20
Vấn đáp - hiểu code của chính mình	15
TỔNG CỘNG	**100**

5.1. Chức năng đầy đủ và chạy đúng - 30 điểm

- 25-30đ: Tất cả chức năng bắt buộc chạy đúng, không bị lỗi, xử lý tốt các trường hợp bất thường
- 15-24đ: Đa số chức năng bắt buộc hoạt động, có vài lỗi nhỏ
- Dưới 15đ: Thiếu nhiều chức năng bắt buộc hoặc chương trình không chạy được

5.2. Tổ chức code và cách đặt tên - 20 điểm

- Chia file hợp lý, tên biến và hàm bằng tiếng Anh, nhất quán xuyên suốt project
- Không có global variable không cần thiết
- Mỗi hàm chỉ làm một việc (single responsibility ở mức cơ bản)
- Header guard (#ifndef) đúng chuẩn
- Comment code những đoạn phức tạp, khó hiểu (nếu có)

5.3. Lịch sử commit trên Git/GitHub - 15 điểm

- Commit đều đặn trong suốt quá trình, tên commit nói rõ làm gì, đóng góp từ bởi tất cả các thành viên

5.4. Demo chương trình chạy thực tế - 20 điểm

- Chạy được thực tế, không bị crash trong 10 phút demo, xử lý được tình huống thông thường

5.5. Vấn đáp - hiểu code của chính mình - 15 điểm

- Mỗi thành viên giải thích được phần mình làm và hiểu tổng thể chương trình hoạt động như thế nào